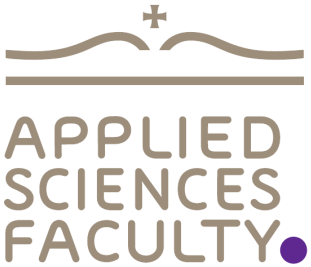
C++ — лекція 8. Препроцесор





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Препроцесор

- Перш ніж потрапити до компілятора, текст програми опрацьовується препроцесором, який згідно своїх інструкцій модифікує його[1].
- Інструкції для препроцесора називаються **директивами**. Вони починаються із символу #, перед яким не може бути нічого, крім пробілів, табуляцій і т.д. — символів-роздільників.

```
#include <somefile.h>
#define SOMEMACRO 1
```

[1] Код програми, тобто. :-)

Символ продовження рядка

- Іноді потрібно створити дуже довгий неперервний рядок. Найчастіше це актуально для наступної теми – макросів.
- Однак, працювати з таким довжелезним рядком буде незручно.
- На такий випадок існує символ продовження рядка.
- Якщо останнім в рядку буде "\", після якого не буде нічого — навіть пробілів[2], то компілятор вважатиме, що розриву рядка немає. Іншими словами — об'єднає цей рядок із наступним.

```
char ls1 [] = "This is very large line \
we have splitted just now.";
```

- Зауважте, що пробіли не відкидатимуться, і код:

```
char ls2 [] = "Here we have \
                    some spaces.";
```

дасть стрічку:

```
"Here we have                    some spaces."
```

[2] C++23 дозволив ігнорувати пробіли після цього символу.

Макроси

Правило 1: Можете **НЕ** користуватися макросами — не користуйтеся.

- Макроси — сутності[3], які під час обробки препроцесором заміняються на вказані стрічки.
- Препроцесор — інструмент доволі примітивний, мало що знає про синтаксис C/C++ і майже нічого про семантику. Тому легко може нашкодити.

[3] Об'єкти в широкому розумінні того слова.

Прості макроси

#define SIMPLE_MACRO text which will be put in place of macro

Там, де в тексті програм зустрічатиметься ідентифікатор, написаний після `#define`, (`SIMPLE_MACRO` в даному випадку), його буде замінено на все, що йде після нього, до кінця рядку. Якщо після імені макросу не йде нічого — він буде замінятися на порожнечу.

- Не використовуйте простих імен для макросів!
- Щоб їх завжди було видно — давайте макросам достатньо помітні, довгі імена. Існує традиція називати їх виключно великими літерами.
- "Відмінити", знищити, макрос можна за допомогою директиви #undef:

  #undef SIMPLE_MACRO

- Іноді такі макроси називають object-like macros.

Макроси з аргументами

- Макроси можуть приймати аргументи:

```
#define MACRO_ARGS(x, y) x+y
```

- При підстановці макросу, передані йому в ролі аргументів стрічки будуть підставлені у відповідних місцях:

```
MACRO_ARGS(a, b); /* Дасть a+b */
MACRO_ARGS("STR1", "STR2"); /* Дасть "STR1"+"STR2" */
```

- Такі макроси іноді називають function-like macros.

Сюрпризи макросів – 1

- Використання макросів має багато сюрпризів!
- Наприклад, нормальний, здавалося б, макрос:

```
#define MACRO_MUL(x, y) x*y
```

Викликаємо:

```
MACRO_MUL(a,b); // a*b. Чудово.
// А так?
MACRO_MUL(a-2,b+3); // Дасть a-2*b+3 --
    //не зовсім те, що очікувалося!
```

Щоб з цим боротися, слід всі аргументи брати в дужки:

```
#define MACRO_MUL_FIXED( x, y) (x)*(y)
  ..........
MACRO_MUL_FIXED(a-2,b+3); // Дасть (a-2)*(b+3) -- OK
```

Сюрпризи макросів – 2

- Нехай є такий макрос, який врахував уроки попереднього:

  #define MACRO_ADD(a,b) (a)+(b)

  Що дасть наступний код?

  z = MACRO_ADD(1,2)*3;

  Правильно:

  z = 1+2*3;

  Не зовсім те, що хотілося.
- Тому весь макрос теж варто брати в дужки:

  #define MACRO_ADD(a,b) ((a)+(b))

  Тоді результат для попереднього виразу стане більш передбачуваним:

  z = (1+2)*3;

Сюрпризи макросів – 3

- Якщо аргументи мають побічні ефекти, це не дуже допомагає.
- Нехай є такий макрос:

  #define MAX(a,b) ((a)>(b)?(a):(b))

  Тоді:

  x = MAX(i++,j++);

  дасть:

  x = ((i++)>(j++)?(i++):(j++));

- З цим теж можна справитися, однак код знову ускладниться[4].
- Альтернативою може бути ретельна документація того факту, що макрос може свої аргументи "обчислювати" декілька раз. Але документацію читають мало і неуважно.
- Краще взагалі утримуватися від підставляння виразів у макроси. Однак – і це головне джерело проблем, макроси з аргументами схожі на функції...

[4] В GCC та CLang, для C, але не для C++, є цікавий макрос __auto_type, див. https://gcc.gnu.org/onlinedocs/gcc/Typeof.html.

Сюрпризи макросів – 4a

- Нехай макрос складається із кількох частин, наприклад, двох викликів функцій:

  #define COMBO_FN_1_AND_2(a,b) fn((a)); fn((b))

  Крапки з комою в кінці немає, щоб зробити його більш схожим на звичайний виклик функції.
- Використати його можна так:

  COMBO_FN_1_AND_2(1,2);

  Все ніби нормально. А якщо так?:

```
if ( condition )
    COMBO_FN_1_AND_2(1,2);
```

  Воно розгорнеться у:

```
if ( condition )
    fn(1);
fn(2);
```

Сюрпризи макросів – 4b

- Навіть якщо зробити так:

  #define COMBO_FN_1_AND_2(a,b) { fn((a)); fn((b)); }

  та проблема зникне, але залишиться інша.

- Код:

```
if ( condition )
    COMBO_FN_1_AND_2(1,2);
else
    other_action(3,4);
```

  розгорнеться в:

```
if ( condition )
{    fn(1); fn(2); };
else
    other_action(3,4);
```

  Зверніть увагу на крапку з комою перед else.

Сюрпризи макросів – 4c

- Вирішити цю проблему допоможе наступна конструкція:

```
#define COMBO_FN_1_AND_2(a,b) \
    do{ fn((a)); fn((b)); }while(false)
```

Використання циклу зробить комбінований макрос єдиним виразом, а false як аргумент do...while приведе до того, що тіло виконається лише раз.

Сюрпризи макросів – 5

- Короткі імена макросів – проблема!
  - Наприклад, головний файл заголовків MS Windows, windows.h історично містить такий макрос:

    #define max(a,b) ((a)>(b)?(a):(b))

    Наслідки уявити не важко.
  - Спосіб вимкнути такі макроси – визначити макрос NOMINMAX. Більший список див. тут.
- Спосіб обійти, для макросів з аргументами – взяти ім'я в дужки:

  (std :: max)(a,b);

Сюрпризи макросів – підсумок

- Увага, макросів є багато інших сюрпризів!
- Зокрема, якщо в макросі помилка, компілятор не може повноцінно вказати місце помилки — він бачить тест програми після препроцесингу. З цим можна боротися, зокрема, попросити компілятор показати код після препроцесингу та аналізувати його. Однак, як показано на попередньому слайді, спроба вирішити одну проблему породжує нову, яку теж потрібно вирішувати, і так може тривати дуже довго.
- Тобто, макроси породжують багато проблем, краще їх, по можливості, уникати. Причиною цього є те, що макроси – занадто грубий засіб, вони нічого не знають про структуру програми – функції, класи, області видимості тощо.
- Увага! Іноді макроси все ж потрібні. Уникання їх не слід перетворювати на релігію!
- Як один із заходів безпеки — покористувалися, відмініть його директивою #undef.

Сюрпризи макросів – посилання

- "Learning to read C++ compiler errors: Nonsensical errors from a function declaration", і, особливо, "The Turkish lira's currency code is an unexpected source of problems with computer programmers" – приклади спричинених макросами неочікуваних проблем.
- ◇ Приклад з посту вище:

```
#pragma push_macro("TRY")
#undef TRY
#include <winrt/Windows.Globalization.h>
#pragma pop_macro("TRY")
```

Засіб нестандартний, але підтримується MSVC, Clang, GCC.

Операції над аргументами – перетворення аргументу в стрічку

- Директива перетворення аргументу в стрічку:

```
#define TO_STRING( txt ) #txt
   ............
 printf (  TO_STRING( Hello World! ) );
/* Дасть       printf (  "Hello  World!" )  */
```

- Однак, якщо аргумент TO_STRING теж макрос, працювати воно не буде. Можна зробити так: Ще один приклад:

```
#define ARG2STR(x) STR(x)
#define STR(x) #x
    ................
#define SIZE 23
 printf ("Size  is  "ARG2STR(SIZE)" bytes\n");
```

Тоді під час першого проходу підставить STR замість ARG2STR, 23 замість SIZE, давши STR(23), тоді наступні проходи згенерують, відповідно, #23 і "23".

Операції над аргументами — конкатенація стрічок аргументів

- Операція конкатенації стрічок аргументів:

```
#define GEN_NEW_NAME(f, s) f##s
    .............
int  GEN_NEW_NAME(var, 11); // Дасть:
                                    // int  var11;
```

- Вразлива до тієї ж проблеми, що і перетворення аргументів у стрічку. Вирішується аналогічно.

Умовна компіляція

- Директива `#ifdef ІДЕНТИФІКАТОР` , якщо ідентифікатор не є оголошеним, пропускає весь текст, поки не зустрінеться `#endif`, `#else` або `#elif`.
- `#if` перевіряє на інші умови, поміж яких може бути і `defined`. Тобто, "`#ifdef SMTH`" еквівалентне "`#if defined SMTH`". Однак, можна будувати і складніші вирази, наприклад:

```
#define SOME_MACRO 7
  .........
#if defined  SOME_MACRO && SOME_MACRO>5
//Some code
#endif
```

- **#elif** дозволяє перевірити ще одну умову.
- **#ifndef** виконується, коли символ не означено.
- **#endif** завершує всі директиви умовної компіляції.
- **#elifdef** (C++23) – аналог `#elif defined`.
- **#elifndef** (C++23) – аналог `#elif !defined`.

Включення файлів

Файл, який включатимемо, include_me.h

```
// Цей файл буде включено.
int var1=55;
char var2='L';

int get_one(){
    return 1;
}
```

Файл, що включає

```
// Включаємо:
#include "include_me.h"

int get_two(){
    return 2*get_one();
}
```

Компілятор побачить

```
// Включаємо:
// Цей файл буде включено.
int var1=55;
char var2='L';

int get_one(){
    return 1;
}

int get_two(){
    return 2*get_one();
}
```

`#include <filename.h>` вказує спершу шукати файл для включення в системних директоріях, лише потім у директорії, де файл. `#include "filename.h"` — навпаки.

Включення двійкових файлів (C++23)

- В C++23 додали директиву #embed, що дозволяє включати двійкові файли безпосередньо:

```
const unsigned char icon_display_data[] = {
                    #embed "art.png"
};

const int please_dont_oom_kill_me[] = {
        #embed "/dev/urandom" limit(512) // Обмежуємо кількість
};

const unsigned char null_terminated_file_data[] = { // Додаємо суфікс
        #embed "might_be_empty.txt" \
                prefix (0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
                suffix (,)                                  // можна також prefix ()
        0 // always null-terminated
};
```

- Відповідна пропозиція до стандарту:
  `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1967r9.html`.

- `#line` — дозволяє керувати виводом інформації про номер рядка та ім'я файлу, де відбулася помилка:

  #line 333 "do it is filename"
  untyped errrrr ;

  повідомить помилку у файлі: "do it is filename", в рядку 333.

- `#error` — генерує помилку із вказаним текстом у вказаному місці.

  #ifndef __cplusplus
  #error A C++ compiler is required!
  #endif

- `#pragma` — директиви, специфічні для компілятора. Якщо компілятор не впізнає конкретну директиву після pragma — він повинен її просто проігнорувати. Див., зокрема, `https://gcc.gnu.org/onlinedocs/gcc/Pragmas.html` – там багато цікавого (наприклад, `#pragma ms_struct`, `#pragma pack(n)`, `#pragma GCC ivdep` для спрощення векторизації, `#pragma GCC diagnostic warning "-Wformat"`, `#pragma GCC poison puts` – заборонити певні символи, `#pragma unroll` вказує розгортати цикли). Однак, обережно – використання pragma може зробити ваш код непортабельним!

- Директива `#warning` – нестандартне розширення GCC, аналогічне `#error`, але створює попередження.

◇ Прийнято до стандарту в C++23.

Визначені наперед макроси

Деякі макроси визначено завжди:

- `__LINE__` — номер поточного рядка.
- `__FILE__` — назва поточного файлу.
- `__DATE__` — стрічка із датою початку компіляції.
- `__TIME__` — стрічка із часом початку компіляції.
- `__cplusplus` — макрос, який вказує, що компілятор — саме C++ (зокрема, на противагу C). Для стандартних компіляторів рівний або більший за 199711L. Зокрема, для: C++11 – 201103L, C++14 – 201402L, C++17 – 201703L.
- `__STDC_HOSTED__` (C++11) — 1, якщо код виконуватиметься під ОС, 0 – якщо ні.
- Приклад:

```
cout << "This file was compiled " << __DATE__;
cout << " at " << __TIME__ << "." << endl;
```

Нестандартні макроси

Різні компілятори, під різними операційними системами визначають багато макросів із інформацією про поточне середовище:

- `MSC_VER` — визначається Visual Studio, містить версію компілятора[5].
- `__GNUC__` — визначається GCC, містить версію компілятора.
- `__linux__` — оголошується деякими компіляторами під час компіляції для Linux.
- `__ANDROID__` — оголошується деякими компіляторами під час компіляції під Android. Так як останній базується на Linux, буде означено також `__linux__`.
- Ще приклади: `__arm__`, `__thumb__` – GCC для ARM, `__i386__` – GCC для 386+, `_M_IX86` – MSVC 32, аналогічно, для x86-64, `__amd64__` – GCC, `_M_AMD64` – MSVC 64, `_POSIX_VERSION`, `_XOPEN_VERSION` – для SUS, `__MINGW32__`. І т.д., і т.п.
- `__COUNTER__` – дає унікальний лічильник, який збільшує своє значення за кожного використання. MSVC, GCC, Clang.
- Доволі детальний перелік див. тут: `http://sourceforge.net/p/predef/wiki/Home/`.

[5] Подробиці щодо значення і визначення версій MSVC: `https://devblogs.microsoft.com/oldnewthing/20221219-00/?p=107601` – "On the large number of ways of expressing Microsoft Visual C++ compiler versions".

```
#ifdef __GNUC__
    cout << "Compiled by GCC" << endl;
    cout << "Version: " << __GNUC__ << '.'
            << __GNUC_MINOR__ << '.'
            << __GNUC_PATCHLEVEL__ << endl;
    #ifdef __MINGW32__
        cout << "Under Windows" << endl;
    #elif defined __linux__
        cout << "Under Linux" << endl;
    #else
        cout << "Under unknown platform" << endl;
    #endif // __MINGW32__
#elif defined _MSC_VER
    cout << "Compiled by Visual studio" << endl;
    cout << "Version: " << _MSC_VER << endl;
#else
    cout << "Compiled by unknown compiler" << endl;
#endif // __GNUC__

cout << "This file was compiled " << __DATE__;
cout << " at " << __TIME__ << "." << endl;
cout << "Source file name: " << __FILE__ << endl;
```

GCC extensions

- `__func__` – макрос із C99, з іменем поточної функції, еквівалентний такому оголошенню:

  ```
  static const char __func__[] = "function-name";
  ```

  `__FUNCTION__` – його старіший синонім, існує для зворотної сумісності.
- `__PRETTY_FUNCTION__` – прототип функції.
- Всі ці сутності ведуть себе як змінні, а не як макроси! Зокрема, ними не можна ініціалізувати C-стрічки чи використовувати в конкатенації стрічкових літералів.
- Модифікований приклад звідси `https://gcc.gnu.org/onlinedocs/gcc/Function-Names.html`:

```
struct C {
  void sub (int i) {
      printf ("__FUNCTION__ = %s\n", __FUNCTION__);
      printf ("__PRETTY_FUNCTION__ = %s\n", __PRETTY_FUN
    }
};
```

Виклик sub() виведе:

```
__FUNCTION__ = sub
__PRETTY_FUNCTION__ = void C::sub(int)
```

std::source_location – C++20

- В C++20 додано більш загальний спосіб аналізувати файли з кодом, що компілюються:

```
void log(const std::string_view message,
         const std::source_location location =
               std::source_location::current())
{
    std::cout << "file: "
              << location.file_name() << "("
              << location.line() << ":"
              << location.column() << ") "
              << location.function_name() << ": "
              << message << '\n';
}
```

- Надає детальнішу інформацію, ніж відповідні макроси і нормально працює з функцій – на відміну від них.
- Відповідний макрос перевірки: __cpp_lib_source_location.
- Есей на тему: "C++20s std::source_location in action".

Feature tests – 1

- Проблема діагностики середовища важлива. Частково її вирішують згадані вище платформо-залежні макроси. Однак, хотілося б більш стандартного рішення.
- Особливо гостро ця проблема стоїть з приводу виходу нових стандартів – різні компілятори в різний час додають підтримку окремих їх можливостей.
- Тому, останнім часом мова підтримує багато так званих feature test macros – макросів тестування можливостей. Макроси ці мають значення, рівне року і місяцю, коли дана можливість була включена в чернетку стандарту.
- Наведемо деякі приклади.
- `__cpp_rtti` та `__cpp_exceptions` – C++98, наявність RTTI та підтрики виключень, відповідно.
- `__cpp_raw_strings`, `__cpp_lambdas`, `__cpp_constexpr`, `__cpp_range_based_for`, `__cpp_rvalue_references`, `__cpp_initializer_lists`, та багато інших – C++11.
- `__cpp_generic_lambdas`, `__cpp_variable_templates` і т.д. і т.п. – C++14.

Feature tests – 2

- `__cpp_inline_variables`, `__cpp_if_constexpr`, `__has_cpp_attribute(fallthrough)`, `__cpp_lib_variant`, `__cpp_lib_parallel_algorithm`, `__cpp_lib_string_view`, `__cpp_lib_filesystem` і десятки інших – C++17, де їх особливо багато.
- Окремо є тести для експериментальних можливостей, що можуть ввійти до наступних стандартів: `__cpp_lib_experimental_net`, `__cpp_concepts`, `__cpp_transactional_memory`, `__cpp_lib_experimental_parallel_algorithm`, `__cpp_lib_experimental_barrier` тощо.
- Детальніше див. `http://en.cppreference.com/w/User:D41D8CD98F/feature_testing_macros` – список, посортований за стандартом та по алфавіту: `http://en.cppreference.com/w/cpp/experimental/feature_test`.
- `https://quuxplusone.github.io/blog/2024/02/06/feature-test-macro-usage-note/` – цікавий есей на тему.

__has_include(header-name)

- В C++17 було додано `__has_include( header-name )` – спосіб перевірити наявність файлу заголовків.
- Цьому макросу можна передавати будь що, що можна передати директиві #include.
- Він рівний 1, якщо такий хідер існує, 0 – якщо ні.
- Зауважте, існування ще не значить працездатності! Наприклад, якийсь заголовок може бути майже порожнім в режимі C++14 і містити все необхідне в режимі C++17.
- Приклад із `http://en.cppreference.com/w/cpp/preprocessor/include`:

```
#if __has_include(<optional>) && __cplusplus > 201402L
#  include <optional>
#  define have_optional 1
#elif __has_include(<experimental/optional>) && \
                                        __cplusplus >= 201402L
#  include <experimental/optional>
#  define have_optional 1
#  define experimental_optional 1
#else
#  define have_optional 0
#endif
```

__has_cpp_attribute(attribute-token)

- `__has_cpp_attribute( attribute-token )` дозволяє перевірити наявність атрибута.
- Розгортається в 0, якщо атрибута немає, в рік-місяць його включення в стандарт для стандартних атрибутів та якесь ненульове значення для нестандартних.
- Приклад:

```
#if !__has_cpp_attribute(noreturn)
#error "Need noreturn!"
#endif
```

Детальніше про препроцесор див., наприклад:

- http://www.cplusplus.com/doc/tutorial/preprocessor/
- Пару статей від Andres Lindgren: http://www.iar.com/Global/Resources/Developers_Toolbox/C_Cplusplus_Programming/Tips%20and%20tricks%20using%20the%20preprocessor%20%28part%20one%29.pdf — "Anders Lindgren — Tips and tricks using the preprocessor (part one)" і http://www.iar.com/Global/Resources/Developers_Toolbox/C_Cplusplus_Programming/Tips%20and%20tricks%20using%20the%20preprocessor%20%28part%20two%29.pdf — part two.
- Стаття від кількох авторів, включаючи Страуструпа, на тему, як нові можливості C++11 дозволяють скоротити використання макросів: "Aditya Kumar, Andrew Sutton, Bjarne Stroustrup — Rejuvenating C++ Programs through Demacrofication", http://www.stroustrup.com/icsm-2012-demacro.pdf
- http://habrahabr.ru/post/246971/ — "Грязные трюки с макросами C++".
- https://quuxplusone.github.io/blog/2021/02/01/x-macros/ – цікавий спосіб використовувати макроси, щоб створювати серії однорідних записів, наприклад, варіантів switch.
- "X-macros: Not all C macros are evil" – не те щоб я з назвою був згоден, але корисні є, факт.

Організація програми — дуже коротко

Оголошення типів, функції, класів, шаблонів, змінних[6], тощо, винесене в окремий файл (часто, із розширенням ".h" або ".hpp"). Їх визначення знаходяться у інших файлах , часто із розширенням ".cpp".

Файл із оголошенням включається (директивою "#include"), а реалізація функцій і т.д., компілюється окремо. Компонувальник об'єднує скомпільовані таким чином файли у кінцеву програму.

Така роздільна компіляція має багато нюансів! Детальніше див. Бьёрн Страуструп, "Язык программирования C++", специальное издание, глава 9 "Исходные файлы и программы".

[6] Нагадаємо, що оголошення змінної, яке не є її визначенням, виглядає як `extern type var;`.

somelib.h

```
#ifndef FUNCTION1_HEADER
#define FUNCTION1_HEADER

typedef int RESULT_T;
RESULT_T function1(int a, int b, double c);

#endif
```

somelib.cpp

```
#include <iostream>
#include "somelib.h"

using namespace std;

RESULT_T function1(int a, int b, double c)
{
    int result=a+b*c;
    cout << "First argument: " << a << endl;
    cout << "First argument: " << b << endl;
    cout << "First argument: " << c << endl;
    return result;
}
```

main.cpp

```
#include <iostream>

using namespace std;

#include "somelib.h"

int main()
{
    RESULT_T function_res;
    function_res=function1(1,2,3);
    cout << res << endl;

    return 0;
}
```

Використання умовної компіляції

- Підтримка різних платформ часто потребує по різному реалізовувати ті чи інші засоби.
- Однак, робити це потрібно ізольовано – не "розмазувати" ifdef-и по всьому коду.
- Справа в тому, що кожен такий ifdef породжує два нових варіанти програми, що потребують тестування.

Приклад

- Добре:

```
#if __GNUC__
    #define my_forceinline __attribute__((always_inline))
#elif defined _MSC_VER
    #define my_forceinline __forceinline
#else // unknown compiler
    #error "Unknown compiler, do not know how to support force inline";
#endif

inline void my_forceinline fn1(){}
inline void my_forceinline fn2(){}
```

Якщо my_forceinline лише бажана, а не критична, замість #error варто зробити:

```
#else // unknown compiler
        #define my_forceinline inline
#endif
```

- Погано:

```
#if __GNUC__
    inline void __attribute__((always_inline)) fn1(){}
#elif defined _MSC_VER
    inline void __forceinline fn1(){}
#else // unknown compiler
    #error "Unknown compiler, do not know how to support force inline";
#endif
#if __GNUC__
    inline void __attribute__((always_inline)) fn2(){}
#elif defined _MSC_VER
    inline void __forceinline fn2(){}
#else // unknown compiler
    #error "Unknown compiler, do not know how to support force inline";
#endif
```

- При всій очевидності його недолугості, через необхідність зусиль для створення відповідних абстракцій, другий підхід часто підступно просочується в код.

Макрос assert

- Макрос assert використовується для перевірки якихось тверджень під час компіляції програми. Якщо твердження невірне — виникає помилка компіляції.

```
assert(some_ptr!=0);
```

- Трюк, щоб додати повідомлення (див. також дискусію тут: `https://stackoverflow.com/questions/3692954/add-custom-messages-in-assert`):

```
assert(some_ptr!=0 && "Pointer should be non-zero");
```

- Означено цей макрос в файлі <cassert>.
- Такий контроль в програмі – надзвичайно потужний інструмент. Однак, будучи реалізованим за допомогою макросів, має всі їх недоліки.
- Оскільки в остаточній версії assert може бути шкідливим, є засіб його вимкнути. Якщо визначено символ NDEBUG до першого включення <cassert> — assert просто не робитиме нічого. (Варто передавати NDEBUG через опції компілятора.)
- Через це категорично не можна в assert робити щось, що впливає на стан програми!
- В C++26 макрос перевизначили приймати __VA_ARGS__ – (...).

static_assert

- assert здійснює перевірку під час виконання – динамічну.
- `static_assert ( bool_constexpr , message )` дозволяє здійснити статичну перевірку – під час компіляції.
- Вираз-умова повинен бути таким, щоб його можна було обчислити під час компіляції – constexpr.
- Якщо умова виконується, не відбувається нічого. Якщо ж ні – стається помилка компіляції та виводиться відповідне повідомлення про помилку:

```
static_assert(sizeof(int)>=4, "Int should have at least 4 bytes");

template<size_t N>
class SidesCont{
    static_assert(N>1 && N<12, "Too many sides!");
};
```

- Повідомлення повинно бути стрічковим літералом.
- До C++20 – зручно використовувати у шаблонах. Дозволяє перевіряти вимоги до типів, з якими вони інстанціюються.
- ◇ В C++20, зазвичай, зручнішими для цього є концепції.

static_assert

- C++17 додав варіант без повідомлення: `static_assert ( bool_constexpr )`.
- C++23 дозволив static_assert(false) не спричиняти помилку компіляції у функціях, що не інстанціюються.
- В C++26 планується додати можливість формування повідомлення про помилку під час компіляції, зокрема – з використання std::format().
- ◇ Приклад див.: "C++26: user-generated static_assert messages".

- Приклад з `en.cppreference.com/w/cpp/language/static_assert`:

```
template <class T>
void swap(T& a, T& b){
    static_assert (std :: is_copy_constructible<T>::value,
                   "Swap requires copying");
    static_assert (std :: is_nothrow_copy_constructible<T>::value
                && std::is_nothrow_copy_assignable<T>::value,
                   "Swap may throw");
    auto c = b;
    b = a;
    a = c;
}

template <class T>
struct data_structure{
    static_assert (std :: is_default_constructible <T>::value,
                   "Data Structure requires default-constructible elements");
};
```

- Зверніть увагу на використання засобами з <type_traits> — is_XXX.

- Post- і pre-conditions, з врахуванням C++20: "GotW #99 Solution: Postconditions", "GotW #97 Solution: Assertions", "GotW #100 Solution: Preconditions, Part 1". "GotW #102 Solution: Assertions and "UB"" – деякі міркування про UB та assert.
- Невизначена поведінка під час компіляції – препроцесор: "Let's enumerate the UB"
  - "C++26: no more UB in lexing" – нарешті виправлено.
  - Прибрано в C++26.
- "Перевантаження" макросів: The "macro overloading" idiom".

